

Object modeling: Object relationships and Communication,

Assisted Problems

Problem 1: Library and Books (Aggregation)

- **Description:** Create a **Library** class that contains multiple **Book** objects. Model the relationship such that a library can have many books, but a book can exist independently (outside of a specific library).
- **Tasks:**
 - Define a **Library** class with an **ArrayList** of **Book** objects.
 - Define a **Book** class with attributes such as **title** and **author**.
 - Demonstrate the aggregation relationship by creating books and adding them to different libraries.
- **Goal:** Understand aggregation by modeling a real-world relationship where the **Library** aggregates **Book** objects.

```
import java.util.Scanner;

class Book {
    //creating variables title and author to indicate
    title and author of book
    String title;
    String author;

    //constructor to initialize values
    public Book(String title, String author) {
        this.title = title;
        this.author = author;
    }

    //method to display details
    public void display() {
        System.out.println("Title : " + this.title);
    }
}
```

```

        System.out.println("Author : " + this.author);
    }
}
class Library {
    private String libraryName;
    private String libraryCode;
    private Book[] books;

    //constructor to initialize values
    public Library(String libraryName, String
libraryCode, Book[] books) {
        this.libraryName = libraryName;
        this.libraryCode = libraryCode;
        this.books = books;
    }
    //method to print details
    public void displayLibrary() {
        System.out.println("\nLibrary name : " +
this.libraryName);
        System.out.println("Library code : " +
this.libraryCode);

        //printing details of books
        for(int i=0; i<books.length; i++) {
            System.out.println("Book " + (i+1) + " is "
+ books[i].title + " and it's author is " +
books[i].author);
        }
    }
}

```

```
public class LibraryAndBooks {
    public static void main(String[] args) {
        //creating Scanner class object
        Scanner input = new Scanner(System.in);

        //taking input for library details
        System.out.println("\nFetching details for a
library.... ");
        System.out.print("Enter library's Name : ");
        String libraryName = input.nextLine();
        System.out.print("Enter library code : ");
        String libraryCode = input.nextLine();
        System.out.print("Enter number of books : ");
        int numberOfBooks = input.nextInt();

        //creating array books to store books and a
variable to indicate index of array
        Book[] books = new Book[numberOfBooks];
        int index = 0;

        //taking input for book details of current
library
        for (int i = 0; i < numberOfBooks; i++) {
            input.nextLine();
            System.out.print("Enter title of book " +
(i+1) + " : ");
            String title = input.nextLine();
            System.out.print("Enter author : ");
            String author = input.nextLine();
        }
    }
}
```

```
//creating object of Book class
Book book = new Book(title, author);

//appending the new book in Book array
books[index] = book;
index++;
}
//creating object of Library class
Library library1 = new Library(libraryName,
libraryCode, books);

//printing details
library1.displayLibrary();

//taking input for library2 details
input.nextLine();
System.out.println("Fetching details for a
library.... ");
System.out.print("Enter library's Name : ");
libraryName = input.nextLine();
System.out.print("Enter library code : ");
libraryCode = input.nextLine();
System.out.print("Enter number of books : ");
numberOfBooks = input.nextInt();

//creating array books to store books and a
variable to indicate index of array
books = new Book[numberOfBooks];
index = 0;
input.nextLine();
```

```
//taking input for book details of current
library
    for (int i = 0; i < numberOfBooks; i++) {
        System.out.print("Enter title of book " +
(i+1) + " : ");
        String title = input.nextLine();
        System.out.print("Enter author : ");
        String author = input.nextLine();

        //creating object of Book class
        Book book = new Book(title, author);

        //appending the new book in Book array
        books[index] = book;
        index++;
    }
    //creating object of Library class
    Library library2 = new Library(libraryName,
libraryCode, books);

    //printing details
    library2.displayLibrary();

    //closing the input
    input.close();
}
}
```

Problem 2: Bank and Account Holders (Association)

- **Description:** Model a relationship where a **Bank** has **Customer** objects associated with it. A **Customer** can have multiple bank accounts, and each account is linked to a **Bank**.
- **Tasks:**
 - Define a **Bank** class and a **Customer** class.
 - Use an association relationship to show that each customer has an account in a bank.
 - Implement methods that enable communication, such as **openAccount()** in the **Bank** class and **viewBalance()** in the **Customer** class.
- **Goal:** Illustrate association by setting up a relationship between customers and the bank.

```
import java.util.ArrayList;
import java.util.List;

// Bank class
class Bank {
    private String name;
    // Association with Customer
    private List<Customer> customers;

    // Constructor to initialize variables
    public Bank(String name) {
        this.name = name;
        this.customers = new ArrayList<>();
    }

    // Method to open a new account for a customer
    public void openAccount(String customerName, String
accountType, double initialBalance) {
        Customer customer =
```

```
findCustomerByName(customerName);
    if (customer == null) {
        // Associating customer with this bank
        customer = new Customer(customerName, this);
        customers.add(customer);
    }
    customer.addAccount(accountType,
initialBalance);
    System.out.println("Account opened successfully
for " + customerName + " with type " + accountType +
".");
}

// Method to find a customer by name
private Customer findCustomerByName(String
customerName) {
    for (Customer customer : customers) {
        if (customer.getName().equals(customerName))
{
            return customer;
        }
    }
    return null;
}

// Method to display all customers and their
accounts
public void displayCustomers() {
    System.out.println("Bank: " + name);
    for (Customer customer : customers) {
```

```
        customer.displayAccounts();
    }
}

// Customer class
class Customer {
    private String name;
    // Association with Bank
    private Bank bank;
    // A customer can have multiple accounts
    private List<Account> accounts;

    // Constructor to initialize Customer
    public Customer(String name, Bank bank) {
        this.name = name;
        this.bank = bank;
        this.accounts = new ArrayList<>();
    }

    // Method to add an account for the customer
    public void addAccount(String accountType, double
initialBalance) {
        accounts.add(new Account(accountType,
initialBalance));
    }

    // Method to view balance of all accounts
    public void displayAccounts() {
        System.out.println("Customer: " + name);
    }
}
```



```
        for (Account account : accounts) {
            System.out.println("  Account Type: " +
account.getAccountType() + ", Balance: " +
account.getBalance());
        }
    }

    // Getter for customer name
    public String getName() {
        return name;
    }
}

// Account class
class Account {
    private String accountType;
    private double balance;

    // Constructor to initialize Account
    public Account(String accountType, double balance) {
        this.accountType = accountType;
        this.balance = balance;
    }

    // Getter for account type
    public String getAccountType() {
        return accountType;
    }

    // Getter for balance
```

```
public double getBalance() {
    return balance;
}

// Main class to test the implementation
public class BankApplication {
    public static void main(String[] args) {
        // Creating a Bank
        Bank bank = new Bank("National Bank");

        // Opening accounts for customers
        bank.openAccount("Alice", "Savings", 1000.0);
        bank.openAccount("Bob", "Current", 2000.0);
        bank.openAccount("Alice", "Current", 1500.0);

        // Displaying customers and their accounts
        bank.displayCustomers();
    }
}
```

Problem 3: Company and Departments (Composition)

- **Description:** A **Company** has several **Department** objects, and each department contains **Employee** objects. Model this using composition, where deleting a company should also delete all departments and employees.
- **Tasks:**
 - Define a **Company** class that contains multiple **Department** objects.
 - Define an **Employee** class within each **Department**.

- Show the composition relationship by ensuring that when a **Company** object is deleted, all associated **Department** and **Employee** objects are also removed.
- **Goal:** Understand composition by implementing a relationship where **Department** and **Employee** objects cannot exist without a **Company**.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Employee class
class Employee {
    private String name;
    private String position;

    // Constructor to initialize Employee
    public Employee(String name, String position) {
        this.name = name;
        this.position = position;
    }

    // method to display details
    public void displayDetails() {
        System.out.println("Employee Name: " + name + ",
Position: " + position);
    }
}

// Department class
class Department {
```

```
private String name;

// Employees belong to a Department
private List<Employee> employees;

// Constructor to initialize Department
public Department(String name) {
    this.name = name;
    this.employees = new ArrayList<>();
}

// Adding an employee to the department
public void addEmployee(String name, String
position) {
    employees.add(new Employee(name, position));
}

// Displaying department details
public void displayDetails() {
    System.out.println("Department Name: " + name);
    System.out.println("Employees:");
    for (Employee employee : employees) {
        employee.displayDetails();
    }
}
}

// Company class
class Company {
    private String name;
```

```
private List<Department> departments; // Departments
belong to the Company

// Constructor to initialize Company
public Company(String name) {
    this.name = name;
    this.departments = new ArrayList<>();
}

// Adding a department to the company
public void addDepartment(Department department) {
    departments.add(department);
}

// Displaying company details
public void displayDetails() {
    System.out.println("\nCompany Name: " + name);
    System.out.println("Departments:");
    for (Department department : departments) {
        department.displayDetails();
    }
}

// Clearing all departments (composition
relationship)
public void deleteCompany() {
    // Removing all departments and employees
    departments.clear();
    System.out.println("Company and all its
departments and employees have been deleted.");
}
```

```

    }
}

// Main class to test the implementation with user
input
public class CompanyApplication {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Creating a Company
        System.out.print("Enter the company name: ");
        String companyName = input.nextLine();
        Company company = new Company(companyName);

        // Adding Departments
        System.out.print("Enter the number of
departments: ");
        int numberOfDepartments = input.nextInt();
        input.nextLine(); // Consume newline character

        for (int i = 1; i <= numberOfDepartments; i++) {
            System.out.print("Enter the name of
Department " + i + ": ");
            String departmentName = input.nextLine();
            Department department = new
Department(departmentName);

            // Adding Employees to the Department
            System.out.print("Enter the number of
employees in " + departmentName + ": ");

```

```
int numberOfEmployees = input.nextInt();
input.nextLine(); // Consume newline
character

for (int j = 1; j <= numberOfEmployees; j++)
{
    System.out.print("Enter the name of
Employee " + j + ": ");
    String employeeName = input.nextLine();
    System.out.print("Enter the position of
Employee " + j + ": ");
    String employeePosition =
input.nextLine();
    department.addEmployee(employeeName,
employeePosition);
}

// Adding Department to Company
company.addDepartment(department);
}

// Displaying Company Details
company.displayDetails();

//deletion of the Company
System.out.println("\nDo you want to delete the
company? (yes/no): ");
String deleteChoice = input.nextLine();
if (deleteChoice.equalsIgnoreCase("yes")) {
    company.deleteCompany();
}
```

```
    } else {  
        System.out.println("Company not deleted.");  
    }  
  
    // Closing the input  
    input.close();  
}  
}
```

Self Problems

Problem 1: School and Students with Courses (Association and Aggregation)

- **Description:** Model a **School** with multiple **Student** objects, where each student can enroll in multiple courses, and each course can have multiple students.
- **Tasks:**
 - Define **School**, **Student**, and **Course** classes.
 - Model an association between **Student** and **Course** to show that students can enroll in multiple courses.
 - Model an aggregation relationship between **School** and **Student**.
 - Demonstrate how a student can view the courses they are enrolled in and how a course can show its enrolled students.
- **Goal:** Practice association by modeling many-to-many relationships between students and courses.

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Scanner;  
  
// Class representing a School  
class School {
```



```
private String name;
private List<Student> students;

//constructor to initialize School
public School(String name) {
    this.name = name;
    this.students = new ArrayList<>();
}
//getter method to get Name
public String getName() {
    return name;
}
//method to add student
public void addStudent(Student student) {
    students.add(student);
}
//method to display student
public void displayStudents() {
    System.out.println("School: " + name);
    System.out.println("Students:");
    for (Student student : students) {
        System.out.println("- " +
student.getName());
    }
}

// Class representing a Student
class Student {
    private String name;
```

```
private List<Course> enrolledCourses;

//constructor to initialize Student
public Student(String name) {
    this.name = name;
    this.enrolledCourses = new ArrayList<>();
}
//getter method to get name
public String getName() {
    return name;
}
//method to enroll course
public void enrollCourse(Course course) {
    enrolledCourses.add(course);
    course.addStudent(this);
}
//method to display details
public void displayEnrolledCourses() {
    System.out.println("Student Name: " + name);
    System.out.println("Enrolled Courses:");
    for (Course course : enrolledCourses) {
        System.out.println("- " +
course.getCourseName());
    }
}

// Class representing a Course
class Course {
    private String courseName;
```

```
private List<Student> enrolledStudents;

//constructor to initialize course
public Course(String courseName) {
    this.courseName = courseName;
    this.enrolledStudents = new ArrayList<>();
}

//getter method to get course name
public String getCourseName() {
    return courseName;
}

//method to add student
public void addStudent(Student student) {
    enrolledStudents.add(student);
}

//method to display enrolled students
public void displayEnrolledStudents() {
    System.out.println("Course: " + courseName);
    System.out.println("Enrolled Students:");
    for (Student student : enrolledStudents) {
        System.out.println("- " +
student.getName());
    }
}

// Main class to test School, Students, and Courses
public class SchoolManagementSystem {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```

```
//taking user input for school details
System.out.print("Enter the name of the school:
");
String schoolName = input.nextLine();
School school = new School(schoolName);

//taking user input for number of students and
create Student objects
System.out.print("Enter the number of students:
");
int numStudents = input.nextInt();
input.nextLine();

List<Student> students = new ArrayList<>();
for (int i = 0; i < numStudents; i++) {
    System.out.print("Enter the name of student
" + (i + 1) + ": ");
    String studentName = input.nextLine();
    Student student = new Student(studentName);
    students.add(student);
    school.addStudent(student);
}

// taking user input for number of courses and
create Course objects
System.out.print("Enter the number of courses:
");
int numCourses = input.nextInt();
input.nextLine();
```

```

List<Course> courses = new ArrayList<>();
for (int i = 0; i < numCourses; i++) {
    System.out.print("Enter the name of course "
+ (i + 1) + ": ");
    String courseName = input.nextLine();
    courses.add(new Course(courseName));
}

// Enrolling students in courses
for (Student student : students) {
    System.out.println("Enroll courses for
student: " + student.getName());
    for (int i = 0; i < courses.size(); i++) {
        System.out.println((i + 1) + ". " +
courses.get(i).getCourseName());
    }
    System.out.print("Enter course numbers
(comma-separated): ");
    String[] courseChoices =
input.nextLine().split(",");
    for (String choice : courseChoices) {
        int courseIndex =
Integer.parseInt(choice.trim()) - 1;
        if (courseIndex >= 0 && courseIndex <
courses.size()) {
            student.enrollCourse(courses.get(courseIndex));
        }
    }
}

```

```

    }

    // Displaying school details
    school.displayStudents();

    // Displaying courses with enrolled students
    for (Course course : courses) {
        course.displayEnrolledStudents();
        System.out.println();
    }

    // Displaying each student's enrolled courses
    for (Student student : students) {
        student.displayEnrolledCourses();
        System.out.println();
    }
    //closing the input
    input.close();
}
}

```

Problem 2: University with Faculties and Departments (Composition and Aggregation)

- Description:** Create a **University** with multiple **Faculty** members and **Department** objects. Model it so that the **University** and its **Departments** are in a composition relationship (deleting a university deletes all departments), and the **Faculty** members are in an aggregation relationship (faculty can exist outside of any specific department).
- Tasks:**
 - Define a **University** class with **Department** and **Faculty** classes.
 - Demonstrate how deleting a **University** also deletes its **Departments**.
 - Show that **Faculty** members can exist independently of a **Department**.

- **Goal:** Understand the differences between composition and aggregation in modeling complex hierarchical relationships.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Class representing a Faculty
class Faculty {
    private String name;

    public Faculty(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void display() {
        System.out.println("Faculty Name: " + name);
    }
}

// Class representing a Department
class Department {
    private String name;
    private List<Faculty> faculties;

    public Department(String name) {
        this.name = name;
    }
}
```

```
        this.faculties = new ArrayList<>();
    }

    public String getName() {
        return name;
    }

    public void addFaculty(Faculty faculty) {
        faculties.add(faculty);
    }

    public void displayFaculties() {
        System.out.println("Department: " + name);
        System.out.println("Faculties:");
        for (Faculty faculty : faculties) {
            faculty.display();
        }
    }
}

// Class representing a University
class University {
    private String name;
    private List<Department> departments;

    public University(String name) {
        this.name = name;
        this.departments = new ArrayList<>();
    }
}
```



```
public void addDepartment(Department department) {
    departments.add(department);
}

public void displayDepartments() {
    System.out.println("University: " + name);
    for (Department department : departments) {
        department.displayFaculties();
        System.out.println();
    }
}

}

// Main class to demonstrate composition and
// aggregation
public class UniversityManagement {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Input university details
        System.out.print("Enter the name of the
university: ");
        String universityName = input.nextLine();
        University university = new
University(universityName);

        // Input number of departments
        System.out.print("Enter the number of
departments: ");
        int numDepartments = input.nextInt();
```

```

input.nextLine();

for (int i = 0; i < numDepartments; i++) {
    // Input department details
    System.out.print("Enter the name of
department " + (i + 1) + ": ");
    String departmentName = input.nextLine();
    Department department = new
Department(departmentName);

    // Input number of faculties in the
department
    System.out.print("Enter the number of
faculties in " + departmentName + ": ");
    int numFaculties = input.nextInt();
    input.nextLine();

    for (int j = 0; j < numFaculties; j++) {
        System.out.print("Enter the name of
faculty " + (j + 1) + ": ");
        String facultyName = input.nextLine();
        Faculty faculty = new
Faculty(facultyName);
        department.addFaculty(faculty);
    }

    university.addDepartment(department);
}

// Display university details

```

```
university.displayDepartments();

// Closing input scanner
input.close();

// Demonstrating composition
System.out.println("\nDeleting the university
also deletes its departments.");
}
}
```

Problem 3: Hospital, Doctors, and Patients (Association and Communication)

- **Description:** Model a **Hospital** where **Doctor** and **Patient** objects interact through consultations. A doctor can see multiple patients, and each patient can consult multiple doctors.
- **Tasks:**
 - Define a **Hospital** class containing **Doctor** and **Patient** classes.
 - Create a method **consult()** in the **Doctor** class to show communication, which would display the consultation between a doctor and a patient.
 - Model an association between doctors and patients to show that doctors and patients can have multiple relationships.
- **Goal:** Practice creating an association with communication between objects by modeling doctor-patient consultations.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Class representing a Patient
class Patient {
```

```
private String name;
private int age;
private List<Doctor> consultedDoctors;

public Patient(String name, int age) {
    this.name = name;
    this.age = age;
    this.consultedDoctors = new ArrayList<>();
}

public String getName() {
    return name;
}

public void addDoctor(Doctor doctor) {
    consultedDoctors.add(doctor);
}

public void displayConsultedDoctors() {
    System.out.println("Patient: " + name);
    System.out.println("Consulted Doctors:");
    for (Doctor doctor : consultedDoctors) {
        System.out.println("- Dr. " +
doctor.getName() + " (" + doctor.getSpecialization() +
")");
    }
}

// Class representing a Doctor
```

```
class Doctor {
    private String name;
    private String specialization;
    private List<Patient> patients;

    public Doctor(String name, String specialization) {
        this.name = name;
        this.specialization = specialization;
        this.patients = new ArrayList<>();
    }

    public String getName() {
        return name;
    }

    public String getSpecialization() {
        return specialization;
    }

    public void consult(Patient patient) {
        patients.add(patient);
        patient.addDoctor(this);
        System.out.println("Dr. " + name + " is consulting patient " + patient.getName());
    }

    public void displayPatients() {
        System.out.println("Dr. " + name + " (" + specialization + ")");
        System.out.println("Patients:");
    }
}
```

```
        for (Patient patient : patients) {
            System.out.println("- " +
patient.getName());
        }
    }
}

// Main class representing a Hospital
public class HospitalManagementSystem {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Input number of doctors
        System.out.print("Enter the number of doctors in
the hospital: ");
        int numDoctors = input.nextInt();
        input.nextLine();

        List<Doctor> doctors = new ArrayList<>();
        for (int i = 0; i < numDoctors; i++) {
            // Input doctor details
            System.out.print("Enter the name of doctor "
+ (i + 1) + ": ");
            String doctorName = input.nextLine();
            System.out.print("Enter the specialization
of Dr. " + doctorName + ": ");
            String specialization = input.nextLine();

            doctors.add(new Doctor(doctorName,
specialization));
        }
    }
}
```

```
}

// Input number of patients
System.out.print("Enter the number of patients
in the hospital: ");
int numPatients = input.nextInt();
input.nextLine();

List<Patient> patients = new ArrayList<>();
for (int i = 0; i < numPatients; i++) {
    // Input patient details
    System.out.print("Enter the name of patient
" + (i + 1) + ": ");
    String patientName = input.nextLine();
    System.out.print("Enter the age of " +
patientName + ": ");
    int age = input.nextInt();
    input.nextLine();

    patients.add(new Patient(patientName, age));
}

// Simulating consultations
System.out.println("\nEnter consultations
(Doctor ID and Patient ID):");
System.out.println("Doctor ID (1 to " +
numDoctors + "), Patient ID (1 to " + numPatients +
")");
System.out.println("Enter -1 to stop.");
while (true) {
```

```
System.out.print("Enter Doctor ID: ");
int doctorId = input.nextInt();
if (doctorId == -1) break;

System.out.print("Enter Patient ID: ");
int patientId = input.nextInt();
if (patientId == -1) break;

if (doctorId >= 1 && doctorId <= numDoctors
&& patientId >= 1 && patientId <= numPatients) {
    Doctor doctor = doctors.get(doctorId -
1);
    Patient patient = patients.get(patientId
- 1);
    doctor.consult(patient);
} else {
    System.out.println("Invalid IDs. Try
again.");
}
}

// Displaying details
System.out.println("\nDoctor Details:");
for (Doctor doctor : doctors) {
    doctor.displayPatients();
    System.out.println();
}

System.out.println("Patient Details:");
for (Patient patient : patients) {
```



```
        patient.displayConsultedDoctors();  
        System.out.println();  
    }  
  
    // Closing input  
    input.close();  
}  
}
```

Problem 4: E-commerce Platform with Orders, Customers, and Products

- **Description:** Design an e-commerce platform with **Order**, **Customer**, and **Product** classes. Model relationships where a **Customer** places an **Order**, and each **Order** contains multiple **Product** objects.
- **Goal:** Show communication and object relationships by designing a system where customers communicate through orders, and orders aggregate products.

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Scanner;  
  
// Class representing a Product  
class Product {  
    private String name;  
    private double price;  
  
    public Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
}
```

```
public String getName() {  
    return name;  
}  
  
public double getPrice() {  
    return price;  
}  
}  
  
// Class representing a Customer  
class Customer {  
    private String name;  
    private String email;  
  
    public Customer(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
  
    public void placeOrder(Order order) {  
        System.out.println("Customer " + name + " has  
placed an order.");  
    }  
}
```

```

        order.displayOrderDetails();
    }
}

// Class representing an Order
class Order {
    private Customer customer;
    private List<Product> products;
    private double totalAmount;

    public Order(Customer customer) {
        this.customer = customer;
        this.products = new ArrayList<>();
        this.totalAmount = 0.0;
    }

    public void addProduct(Product product) {
        products.add(product);
        totalAmount += product.getPrice();
    }

    public void displayOrderDetails() {
        System.out.println("\nOrder Details:");
        System.out.println("Customer: " +
customer.getName() + " (" + customer.getEmail() + ")");
        System.out.println("Products Ordered:");
        for (Product product : products) {
            System.out.println("- " + product.getName()
+ " ($" + product.getPrice() + ")");
        }
    }
}

```

```
        System.out.println("Total Amount: $" +
totalAmount);
    }

    public double getTotalAmount() {
        return totalAmount;
    }
}

// Main class to simulate the E-commerce System
public class EcommercePlatform {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Input customer details
        System.out.print("Enter customer name: ");
        String customerName = input.nextLine();
        System.out.print("Enter customer email: ");
        String customerEmail = input.nextLine();

        // Creating a customer object
        Customer customer = new Customer(customerName,
customerEmail);

        // Input number of products available for the
order
        System.out.print("Enter the number of products
available: ");
        int numProducts = input.nextInt();
        input.nextLine(); // to clear the buffer
```

```

        List<Product> availableProducts = new
ArrayList<>();
        for (int i = 0; i < numProducts; i++) {
            System.out.print("Enter product name " + (i
+ 1) + ": ");
            String productName = input.nextLine();
            System.out.print("Enter product price: ");
            double productPrice = input.nextDouble();
            input.nextLine(); // to clear the buffer

            Product product = new Product(productName,
productPrice);
            availableProducts.add(product);
        }

        // Creating an order object
        Order order = new Order(customer);

        // Input number of products customer wants to
order

        System.out.print("Enter the number of products
the customer wants to order: ");
        int numOrderProducts = input.nextInt();
        input.nextLine(); // to clear the buffer

        // Add products to the order
        for (int i = 0; i < numOrderProducts; i++) {
            System.out.print("Enter product number (1 to
" + numProducts + ") for the order: ");

```

```
        int productNumber = input.nextInt();
        input.nextLine(); // to clear the buffer

        if (productNumber >= 1 && productNumber <=
numProducts) {

order.addProduct(availableProducts.get(productNumber -
1));
        } else {
            System.out.println("Invalid product
number. Try again.");
            i--;
        }
    }

    // Customer places the order
    customer.placeOrder(order);

    // Closing input scanner
    input.close();
}
}
```

Problem 5: University Management System

- **Description:** Model a university system with **Student**, **Professor**, and **Course** classes. Students enroll in courses, and professors teach courses. Ensure students and professors can communicate through methods like **enrollCourse()** and **assignProfessor()**.
- **Goal:** Use association and aggregation to create a university system that emphasizes relationships and interactions among students, professors, and courses.

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

// Class Student
class Student {
    private String name;
    private List<Course> enrolledCourses;

    //constructor to initialize Student
    public Student(String name) {
        this.name = name;
        this.enrolledCourses = new ArrayList<>();
    }
    //getter method to get name
    public String getName() {
        return name;
    }
    //method to enroll course
    public void enrollCourse(Course course) {
        enrolledCourses.add(course);
        course.addStudent(this);
    }
    //method to display details
    public void displayCourses() {
        System.out.println("Student Name: " + name);
        System.out.println("Enrolled Courses:");
        for (Course course : enrolledCourses) {
            System.out.println("- " +
```

```
course.getCourseName());
    }
}

// Class Professor
class Professor {
    private String name;
    private List<Course> assignedCourses;

    //constructor to initialize Professor
    public Professor(String name) {
        this.name = name;
        this.assignedCourses = new ArrayList<>();
    }
    //getter method to get name
    public String getName() {
        return name;
    }
    //method to assign course
    public void assignCourse(Course course) {
        assignedCourses.add(course);
        course.setProfessor(this);
    }
    //method to display details
    public void displayCourses() {
        System.out.println("Professor Name: " + name);
        System.out.println("Assigned Courses:");
        for (Course course : assignedCourses) {
            System.out.println("- " +
```



```
course.getCourseName());
    }
}

// Class Course
class Course {
    private String courseName;
    private Professor professor;
    private List<Student> enrolledStudents;

    //constructor to initialize Course
    public Course(String courseName) {
        this.courseName = courseName;
        this.enrolledStudents = new ArrayList<>();
    }
    //getter method to get course name
    public String getCourseName() {
        return courseName;
    }
    //method to set professor
    public void setProfessor(Professor professor) {
        this.professor = professor;
    }
    //method to add student
    public void addStudent(Student student) {
        enrolledStudents.add(student);
    }
    //method to display details
    public void displayCourseDetails() {
```

```
        System.out.println("Course Name: " +
courseName);
        System.out.println("Professor: " + (professor !=
null ? professor.getName() : "None Assigned"));
        System.out.println("Enrolled Students:");
        for (Student student : enrolledStudents) {
            System.out.println("- " +
student.getName());
        }
    }
}

// Main class to test the University Management System
public class UniversityManagementSystem {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // Creating lists to manage entities
        List<Student> students = new ArrayList<>();
        List<Professor> professors = new ArrayList<>();
        List<Course> courses = new ArrayList<>();

        //taking input for number of students and
creating Student objects
        System.out.print("Enter the number of students:
");

        int numStudents = input.nextInt();
        input.nextLine();

        for (int i = 0; i < numStudents; i++) {
```

```

        System.out.print("Enter the name of student
" + (i + 1) + ": ");
        String name = input.nextLine();
        students.add(new Student(name));
    }

    //taking input for number of professors and
    create Professor objects
    System.out.print("Enter the number of
professors: ");
    int numProfessors = input.nextInt();
    input.nextLine();

    for (int i = 0; i < numProfessors; i++) {
        System.out.print("Enter the name of
professor " + (i + 1) + ": ");
        String name = input.nextLine();
        professors.add(new Professor(name));
    }

    // taking input for number of courses and create
    Course objects
    System.out.print("Enter the number of courses:
");
    int numCourses = input.nextInt();
    input.nextLine();

    for (int i = 0; i < numCourses; i++) {
        System.out.print("Enter the name of course "
+ (i + 1) + ": ");

```

```
String courseName = input.nextLine();
courses.add(new Course(courseName));
}

// Assigning professors to courses
for (Course course : courses) {
    System.out.println("Assign a professor to
course: " + course.getCourseName());
    for (int i = 0; i < professors.size(); i++)
    {
        System.out.println((i + 1) + ". " +
professors.get(i).getName());
    }
    //taking user input for professor number
    System.out.print("Enter professor number :
");

    int choice = input.nextInt();
    input.nextLine();
    if (choice > 0 && choice <=
professors.size()) {
        Professor selectedProfessor =
professors.get(choice - 1);
        selectedProfessor.assignCourse(course);
    }
}

// Enrolling students in courses
for (Student student : students) {
    System.out.println("Enroll courses for
student: " + student.getName());
```

```
        for (int i = 0; i < courses.size(); i++) {
            System.out.println((i + 1) + ". " +
courses.get(i).getCourseName());
        }
        System.out.print("Enter course numbers
(comma-separated): ");
        String[] courseChoices =
input.nextLine().split(",");
        for (String choice : courseChoices) {
            int courseIndex =
Integer.parseInt(choice.trim()) - 1;
            if (courseIndex >= 0 && courseIndex <
courses.size()) {
student.enrollCourse(courses.get(courseIndex));
            }
        }
    }

    // Displaying details of all courses,
professors, and students
    System.out.println("\nUniversity Management
System Details:");
    for (Course course : courses) {
        course.displayCourseDetails();
        System.out.println();
    }
    //closing the input
    input.close();
}
```

CodInClub

Powered By 
BridgeLabz

}